

TUGAS PRAKTIKUM 5
“SINKRONISASI SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Nama : Muhammad Fahrur Rohman
NIM : 2410651056
Kelas : Kamis-C

UNIVERSITAS MUHAMMADIYAH JEMBER
FAKULTAS TEKNIK
PRODI TEKNIK INFORMATIKA
2025-2026

PROGRAM 1 — DEADLOCK VERSION

(dining_deadlock.c)

Langkah Langkah:

Buatlah sebuah file untuk isi program bernama nano dining_deadlock.c denga nisi kode ada pada gambar dibawah ini, lalu compile dan jalankan program sehingga hasilnya nampak seperti berikut:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <stdarg.h>

#define N 5

pthread_mutex_t forks[N];
pthread_mutex_t print_lock;
pthread_barrier_t barrier;
volatile int running = 1;
int eat_count[N];

void msleep(long ms) { usleep(ms * 1000); }

void print_ts(const char *fmt, ...) {
    va_list ap;
    char buf[64];
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    strftime(buf, sizeof(buf), "%H:%M:%S", tm);

    pthread_mutex_lock(&print_lock);
    printf("[%s] ", buf);
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    fflush(stdout);
    pthread_mutex_unlock(&print_lock);
}

void* philosopher(void* arg) {
    int id = *(int*)arg;
    free(arg);

    // jangan randomize supaya timing konsisten
    while (running) {
        print_ts("🍴 Filsuf-%d berpikir singkat...\n", id+1);
        msleep(50);

        print_ts("🍴 Filsuf-%d siap lapar, menunggu semua siap (barrier)...\n", id+1);

        // Sinkronisasi: semua filsuf menunggu di barrier
        pthread_barrier_wait(&barrier);

        // Setelah semua tiba di sini, mereka langsung mencoba ambil sumpit kiri
        int left = id;
        int right = (id + 1) % N;

        pthread_mutex_lock(&forks[left]);
        print_ts("// Filsuf-%d mengambil sumpit kiri (%d)\n", id+1, left);

        // disini kita beri sedikit delay agar semua sempat mengambil left
        msleep(200);
```

```

        // kemudian tiap filsuf mencoba ambil kanan -> deadlock kemungkinan besar terjadi
        print_ts(" // Filsuf-%d mencoba ambil sumpit kanan (%d)\n", id+1, right);
        pthread_mutex_lock(&forks[right]);
        print_ts(" // Filsuf-%d mengambil sumpit kanan (%d)\n", id+1, right);

        // makan jika berhasil (tapi di versi paksa ini biasanya tidak sampai sini)
        print_ts(" 🍴 Filsuf-%d makan...\n", id+1);
        eat_count[id]++;
        msleep(100);

        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);

        msleep(50);
    }
    return NULL;
}

int main(int argc, char **argv) {
    int run_seconds = 10; // waktu singkat cukup untuk demo deadlock
    if (argc >= 2) run_seconds = atoi(argv[1]);

    pthread_t th[N];
    pthread_mutex_init(&print_lock, NULL);
    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        eat_count[i] = 0;
    }
    pthread_barrier_init(&barrier, NULL, N);

    print_ts("=== DINING PHILOSOPHERS: FORCED DEADLOCK VERSION ===\n");
    for (int i = 0; i < N; i++) {
        int *id = malloc(sizeof(int));
        if (!id) { perror("malloc"); exit(1); }
        *id = i;
        if (pthread_create(&th[i], NULL, philosopher, id) != 0) {
            perror("pthread_create");
            exit(1);
        }
    }

    // jalankan beberapa detik lalu stop
    sleep(run_seconds);
    running = 0;

    for (int i = 0; i < N; i++) pthread_join(th[i], NULL);

    print_ts("\n=== STATISTIK (%d detik) ===\n", run_seconds);
    for (int i = 0; i < N; i++) {
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
    }

    pthread_barrier_destroy(&barrier);
    for (int i = 0; i < N; i++) pthread_mutex_destroy(&forks[i]);
    pthread_mutex_destroy(&print_lock);
    return 0;
}

```

```

fahru@FAHRUR:~/praktikum5/tugas$ nano dining_deadlock.c
fahru@FAHRUR:~/praktikum5/tugas$ nano dining_resource_ordering.c
fahru@FAHRUR:~/praktikum5/tugas$ nano dining_limited.c
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_deadlock.c -o dining_deadlock -pthread
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_resource_ordering.c -o dining_resource_ordering -pthread
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_limited.c -o dining_limited -pthread

```

Hasil

```
[20:51:43] 🤔 Filsuf-1 berpikir singkat...
[20:51:43] 🤔 Filsuf-2 berpikir singkat...
[20:51:43] 🤔 Filsuf-3 berpikir singkat...
[20:51:43] 🤔 Filsuf-4 berpikir singkat...
[20:51:43] 🤔 Filsuf-5 berpikir singkat...
[20:51:43] 🍽️ Filsuf-1 siap lapar, menunggu semua siap (barrier)...
[20:51:43] 🍽️ Filsuf-3 siap lapar, menunggu semua siap (barrier)...
[20:51:43] 🍽️ Filsuf-5 siap lapar, menunggu semua siap (barrier)...
[20:51:43] 🍽️ Filsuf-4 siap lapar, menunggu semua siap (barrier)...
[20:51:43] 🍽️ Filsuf-2 siap lapar, menunggu semua siap (barrier)...
[20:51:43] // Filsuf-2 mengambil sumpit kiri (1)
[20:51:43] // Filsuf-1 mengambil sumpit kiri (0)
[20:51:43] // Filsuf-4 mengambil sumpit kiri (3)
[20:51:43] // Filsuf-5 mengambil sumpit kiri (4)
[20:51:43] // Filsuf-3 mengambil sumpit kiri (2)
[20:51:43] // Filsuf-4 mencoba ambil sumpit kanan (4)
[20:51:43] // Filsuf-1 mencoba ambil sumpit kanan (1)
[20:51:43] // Filsuf-5 mencoba ambil sumpit kanan (0)
[20:51:43] // Filsuf-3 mencoba ambil sumpit kanan (3)
[20:51:43] // Filsuf-2 mencoba ambil sumpit kanan (2)
^C
fahru@FAHRUR:~/praktikum5/tugas$ |
```

Bisa kita lihat berdasarkan gambar diatas bahwa setiap filsuf mengambil sumpit kiri lalu menunggu kanan sehingga terbentuk siklus saling menunggu (1 menunggu 2, 2 menunggu 3, ..., 5 menunggu 1) dan tidak ada yang dapat melanjutkan yang mengakibatkan program menjadi hang.

PROGRAM 2 RESOURCE ORDERING (dining_resource_ordering.c) (Solusi 1 untuk deadlock)

Langkah Langkah:

Buatlah sebuah file untuk isi program bernama dining_resource_ordering.c dengan isi kodenya seperti gambar dibawah, lalu compile dan jalankan selama 60 detik sehingga hasilnya seperti pada gambar dibawah gambar kode berikut ini:

```
// dining_resource_ordering.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <stdarg.h>

#define N 5

pthread_mutex_t forks[N];
pthread_mutex_t print_lock;
volatile int running = 1;
int eat_count[N];

void msleep(long ms){ usleep(ms * 1000); }

void print_ts(const char *fmt, ...){
    va_list ap;
    char buf[64];
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    strftime(buf, sizeof(buf), "%H:%M:%S", tm);

    pthread_mutex_lock(&print_lock);
    printf("[%s] ", buf);
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    fflush(stdout);
    pthread_mutex_unlock(&print_lock);
}

void* philosopher(void* arg){
    int id = *(int*)arg;
    free(arg);

    srand(time(NULL) + id * 17);

    while (running) {
        print_ts("🤖 Filsuf-%d sedang berpikir...\n", id+1);
        msleep(100 + rand()%900);

        print_ts("🍴 Filsuf-%d lapar, mencoba ambil sumpit...\n", id+1);

        int left = id;
        int right = (id + 1) % N;

        // pick in ordered manner: smaller index first
        int first = left < right ? left : right;
        int second = left < right ? right : left;

        pthread_mutex_lock(&forks[first]);
        print_ts("🍴 Filsuf-%d mengambil sumpit (%d)\n", id+1, first);
        msleep(30);
        pthread_mutex_lock(&forks[second]);
        print_ts("🍴 Filsuf-%d mengambil sumpit (%d)\n", id+1, second);

        print_ts("🍽 Filsuf-%d sedang MAKAN...\n", id+1);
        eat_count[id]++;
    }
}
```

```

        msleep(100 + rand()%300);

        pthread_mutex_unlock(&forks[second]);
        pthread_mutex_unlock(&forks[first]);
        print_ts(" // Filsuf-%d menaruh kedua sumpit\n", id+1);

        msleep(50);
    }
    return NULL;
}

int main(int argc, char **argv){
    int run_seconds = 60;
    if (argc >= 2) run_seconds = atoi(argv[1]);

    pthread_t th[N];
    pthread_mutex_init(&print_lock, NULL);
    for (int i=0; i<N; i++){
        pthread_mutex_init(&forks[i], NULL);
        eat_count[i]=0;
    }

    print_ts("=== DINING PHILOSOPHERS: RESOURCE ORDERING ===\n");
    for (int i=0; i<N; i++){
        int *id = malloc(sizeof(int)); *id = i;
        pthread_create(&th[i], NULL, philosopher, id);
    }

    sleep(run_seconds);
    running = 0;

    for (int i=0; i<N; i++) pthread_join(th[i], NULL);

    print_ts("\n=== STATISTIK (%d detik) ===\n", run_seconds);
    int total=0;
    for (int i=0; i<N; i++){
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
        total += eat_count[i];
    }
    printf("Total makan: %d\n", total);

    for (int i=0; i<N; i++) pthread_mutex_destroy(&forks[i]);
    pthread_mutex_destroy(&print_lock);
    return 0;
}

```

```

fahru@FAHRUR:~/praktikum5/tugas$ nano dining_deadlock.c
fahru@FAHRUR:~/praktikum5/tugas$ nano dining_resource_ordering.c
fahru@FAHRUR:~/praktikum5/tugas$ nano dining_limited.c
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_deadlock.c -o dining_deadlock -pthread
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_resource_ordering.c -o dining_resource_ordering -pthread
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_limited.c -o dining_limited -pthread

```

Hasil:

```
[19:54:33] // Filsuf-2 menaruh kedua sumpit
[19:54:34] 🍷Filsuf-4 lapar, mencoba ambil sumpit...
[19:54:34] // Filsuf-4 mengambil sumpit (3)
[19:54:34] 🍷Filsuf-3 lapar, mencoba ambil sumpit...
[19:54:34] // Filsuf-3 mengambil sumpit (2)
[19:54:34] // Filsuf-4 mengambil sumpit (4)
[19:54:34] 🍽️ Filsuf-4 sedang MAKAN...
[19:54:34] 🍷Filsuf-5 lapar, mencoba ambil sumpit...
[19:54:34] // Filsuf-5 mengambil sumpit (0)
[19:54:34] 🍷Filsuf-1 lapar, mencoba ambil sumpit...
[19:54:34] // Filsuf-4 menaruh kedua sumpit
[19:54:34] // Filsuf-5 mengambil sumpit (4)
[19:54:34] 🍽️ Filsuf-5 sedang MAKAN...
[19:54:34] // Filsuf-3 mengambil sumpit (3)
[19:54:34] 🍽️ Filsuf-3 sedang MAKAN...
[19:54:34] // Filsuf-5 menaruh kedua sumpit
[19:54:34] // Filsuf-1 mengambil sumpit (0)
[19:54:34] // Filsuf-3 menaruh kedua sumpit
[19:54:34] // Filsuf-1 mengambil sumpit (1)
[19:54:34] 🍽️ Filsuf-1 sedang MAKAN...
[19:54:35] // Filsuf-1 menaruh kedua sumpit
[19:54:35]
=== STATISTIK (60 detik) ===
Filsuf-1: Makan 56 kali
Filsuf-2: Makan 60 kali
Filsuf-3: Makan 62 kali
Filsuf-4: Makan 65 kali
Filsuf-5: Makan 63 kali
Total makan: 306
fahru@FAHRUR:~/praktikum5/tugas$ |
```

Bisa kita lihat berdasarkan gambar diatas yang menggunakan solusi Resource Ordering, setiap filsuf mengambil sumpit dalam urutan yang sama berdasarkan nomor terkecil terlebih dahulu. Dengan cara ini, tidak akan terbentuk siklus saling tunggu sehingga deadlock dapat dihindari sepenuhnya. Program berjalan stabil dan semua filsuf dapat makan bergantian tanpa macet.

PROGRAM 3 — LIMITING PHILOSOPHERS (dining_limited.c)
SEMAPHORE (Mengatur Akses Terlambat)

Langkah Langkah:

Buatlah sebuah file untuk isi program bernama dining_limited.c dengan isi kodenya seperti gambar dibawah, lalu compile dan jalankan selama 60 detik sehingga hasilnya seperti pada gambar dibawah gambar kode berikut ini:


```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <syslog.h>

#define N 5

pthread_mutex_t forks[N];
pthread_mutex_t print_lock;
volatile int running = 1;
int eat_count[N];
sem_t room; // membatasi seberapa banyak filsuf yang punya kesempatan mengambil garpu

void msleep(long ms){ usleep(ms * 1000); }

void print_ts(const char *fmt, ...){
    va_list ap;
    char buf[64];
    time_t t = time(NULL);
    struct tm *tm = localtime(&t);
    strftime(buf, sizeof(buf), "%H:%M:%S", tm);

    pthread_mutex_lock(&print_lock);
    printf("[%s] ", buf);
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    fflush(stdout);
    pthread_mutex_unlock(&print_lock);
}

void* philosopher(void* arg){
    int id = *(int*)arg;
    free(arg);

    srand(time(NULL) + id * 19);

    while (running) {
        print_ts("🤖 Filsuf-%d sedang berpikir...\n", id+1);
        msleep(100 + rand()%900);

        print_ts("🍴 Filsuf-%d lapar, menunggu kesempatan masuk meja (semaphore)...\n", id+1);

        // membatasi jumlah filsuf yang bisa coba mengambil garpu
        sem_wait(&room);

        int left = id;
        int right = (id + 1) % N;

        // mengambil garpu dengan aman (tak dibutuhkan jaminan ucutan karena ukuran ruang mencegah deadlock)
        pthread_mutex_lock(&forks[left]);
        print_ts("🍴 Filsuf-%d mengambil sumpit kiri (%d)\n", id+1, left);
        msleep(30);
        pthread_mutex_lock(&forks[right]);
        print_ts("🍴 Filsuf-%d mengambil sumpit kanan (%d)\n", id+1, right);

        print_ts("🍴 Filsuf-%d sedang MAKAN...\n", id+1);
        eat_count[id]++;
        msleep(100 + rand()%300);

        pthread_mutex_unlock(&forks[right]);
        pthread_mutex_unlock(&forks[left]);
        print_ts("🍴 Filsuf-%d menaruh kedua sumpit\n", id+1);

        // tinggalkan ruang supaya filsuf lain bisa masuk
        sem_post(&room);

        msleep(50);
    }
    return NULL;
}

int main(int argc, char **argv){
    int run_seconds = 60;
    if (argc >= 2) run_seconds = atoi(argv[1]);

    pthread_t th[N];
    pthread_mutex_init(&print_lock, NULL);
    for (int i=0; i<N; i++){
        pthread_mutex_init(&forks[i], NULL);
        eat_count[i]=0;
    }

    // beri izin kebanyakan filsuf N-1 untuk coba mengambil garpu bersamaan (mencegah cycle)
    sem_init(&room, 0, N-1);

    print_ts("=== DINING PHILOSOPHERS: LIMITING PHILOSOPHERS (max %d) ===\n", N-1);
    for (int i=0; i<N; i++){
        int *id = malloc(sizeof(int)); *id = i;
        pthread_create(&th[i], NULL, philosopher, id);
    }

    sleep(run_seconds);
    running = 0;

    for (int i=0; i<N; i++) pthread_join(th[i], NULL);

    print_ts("\n=== STATISTIK (%d detik) ===\n", run_seconds);
    int total=0;
    for (int i=0; i<N; i++){
        printf("Filsuf-%d: Makan %d kali\n", i+1, eat_count[i]);
        total += eat_count[i];
    }
    printf("Total makan: %d\n", total);

    sem_destroy(&room);
    for (int i=0; i<N; i++) pthread_mutex_destroy(&forks[i]);
    pthread_mutex_destroy(&print_lock);
    return 0;
}

```

```
fahru@FAHRUR:~/praktikum5/tugas$ nano dining_deadlock.c
fahru@FAHRUR:~/praktikum5/tugas$ nano dining_resource_ordering.c
fahru@FAHRUR:~/praktikum5/tugas$ nano dining_limited.c
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_deadlock.c -o dining_deadlock -pthread
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_resource_ordering.c -o dining_resource_ordering -pthread
fahru@FAHRUR:~/praktikum5/tugas$ gcc dining_limited.c -o dining_limited -pthread
```

Hasil:

```
[19:56:45] 🍲 Filsuf-5 lapar, menunggu kesempatan masuk meja (semaphore)...
[19:56:46] // Filsuf-1 menaruh kedua sumpit
[19:56:46] 🤔 Filsuf-1 sedang berpikir...
[19:56:46] // Filsuf-4 menaruh kedua sumpit
[19:56:46] // Filsuf-5 mengambil sumpit kiri (4)
[19:56:46] // Filsuf-3 mengambil sumpit kanan (3)
[19:56:46] 🍲 Filsuf-3 sedang MAKAN...
[19:56:46] // Filsuf-5 mengambil sumpit kanan (0)
[19:56:46] 🍲 Filsuf-5 sedang MAKAN...
[19:56:46] // Filsuf-5 menaruh kedua sumpit
[19:56:46] // Filsuf-3 menaruh kedua sumpit
[19:56:46] 🍲 Filsuf-2 lapar, menunggu kesempatan masuk meja (semaphore)...
[19:56:46] // Filsuf-2 mengambil sumpit kiri (1)
[19:56:46] 🍲 Filsuf-1 lapar, menunggu kesempatan masuk meja (semaphore)...
[19:56:46] // Filsuf-1 mengambil sumpit kiri (0)
[19:56:46] // Filsuf-2 mengambil sumpit kanan (2)
[19:56:46] 🍲 Filsuf-2 sedang MAKAN...
[19:56:47] // Filsuf-2 menaruh kedua sumpit
[19:56:47] // Filsuf-1 mengambil sumpit kanan (1)
[19:56:47] 🍲 Filsuf-1 sedang MAKAN...
[19:56:47] // Filsuf-1 menaruh kedua sumpit
[19:56:47]
=== STATISTIK (60 detik) ===
Filsuf-1: Makan 58 kali
Filsuf-2: Makan 63 kali
Filsuf-3: Makan 55 kali
Filsuf-4: Makan 64 kali
Filsuf-5: Makan 58 kali
Total makan: 298
fahru@FAHRUR:~/praktikum5/tugas$ |
```

Bisa kita lihat berdasarkan gambar diatas yang menggunakan Solusi Limiting Philosophers ia menggunakan semaphore untuk membatasi jumlah filsuf yang boleh mencoba mengambil sumpit secara bersamaan.

Nilai semaphore diinisialisasi sebanyak $N-1$, artinya hanya empat filsuf (dari lima) yang diizinkan berada dalam kondisi “lapar” sekaligus. Dengan pembatasan ini, satu filsuf selalu menunggu di luar, sehingga tidak mungkin terbentuk siklus saling menunggu yang menyebabkan deadlock. Hasil pengujian menunjukkan bahwa program berjalan terus tanpa hang, dan semua filsuf dapat makan bergantian dengan adil.

TABEL EFISENSI/FAIRNESS

Solusi \ Filsuf	F1 F2 F3 F4 F5 Total
ResourceOrdering	56 60 62 65 63 306
Limiting	58 63 55 64 58 298

Berdasarkan tabel diatas, dari kedua pendekatan yang diuji, solusi “Limiting Philosophers” merupakan metode yang paling adil (fair) karena semua filsuf mendapatkan kesempatan untuk makan secara bergantian tanpa mengalami kelaparan (starvation). sementara pada solusi Resource Ordering, walaupun deadlock dapat dicegah, terdapat kemungkinan beberapa filsuf terus kalah cepat mengambil sumpit dan makan lebih sedikit.